Crear un procedimiento almacenado con JDBC

En PostgreSQL puedes utilizar PL/pgSQL para definir el procedimiento y JDBC para enviarlo a la base de datos.

1. Crear el procedimiento en PostgreSQL con JDBC

Adaptamos tu procedimiento aumentar_salario a PostgreSQL:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.SQLException;
public class CrearProcedimiento {
  public static void main(String[] args) {
    String url = "jdbc:postgresql://localhost:5432/empresa";
    String usuario = "postgres";
    String password = "tu_password";
    String sql = """
      CREATE OR REPLACE PROCEDURE aumentar_salario(
        IN p_emp_id INTEGER,
        IN p_porcentaje NUMERIC
      )
      LANGUAGE plpgsql
      AS $$
      BEGIN
        IF p_porcentaje <= 0 THEN
          RAISE EXCEPTION
            'El porcentaje debe ser mayor que cero.';
```

```
            END IF;
            UPDATE empleados
            SET salario = salario *
              (1 + p_porcentaje / 100)
            WHERE emp_id = p_emp_id;
            IF NOT FOUND THEN
              RAISE EXCEPTION
                'Empleado no encontrado.';
            END IF;
          END;
          $$;
          """;
      try (Connection con =
            DriverManager.getConnection(url, usuario, password);
          Statement st = con.createStatement()) {
          st.execute(sql);
          System.out.println(
            "Procedimiento creado correctamente."
          );
      } catch (SQLException e) {
          System.err.println(e.getMessage());
      }
  }
}
```

1.1 Diferencias respecto a Oracle

Oracle	**PostgreSQL**
NUMBER	NUMERIC o INTEGER
RAISE_APPLICATION_ERROR	RAISE EXCEPTION
SQL%ROWCOUNT	GET DIAGNOSTICS o FOUND
CREATE OR REPLACE PROCEDURE	CREATE OR REPLACE PROCEDURE
Bloque PL/SQL	LANGUAGE plpgsql
No necesita delimitador propio en el cuerpo	Se suele utilizar $$ ... $$

En PostgreSQL, FOUND indica si la última sentencia relevante encontró o modificó alguna fila. En este ejemplo, permite detectar que el empleado no existe.

2. Ejecutar el procedimiento desde JDBC

Una vez creado, puedes invocarlo mediante CallableStatement:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.CallableStatement;
import java.sql.SQLException;
public class EjecutarProcedimiento {
   public static void main(String[] args) {
      String url = "jdbc:postgresql://localhost:5432/empresa";
      String usuario = "postgres";
      String password = "tu_password";
      String sql = "{call aumentar_salario(?, ?)}";
      try (Connection con =
            DriverManager.getConnection(url, usuario, password);
```

```
        CallableStatement cs = con.prepareCall(sql)) {
        cs.setInt(1, 101);
        cs.setBigDecimal(
          2, new java.math.BigDecimal("10.0")
        );
        cs.execute();
        System.out.println("Salario actualizado.");
      } catch (SQLException e) {
        System.err.println(
          "Error: " + e.getMessage()
        );
      }
    }
}
```

En este ejemplo se aumenta un 10 % el salario del empleado con identificador 101.

3. ¿Qué controlador JDBC necesitas?

Debes añadir el controlador oficial de PostgreSQL, llamado pgJDBC.

Si utilizas Maven, añade esta dependencia al archivo pom.xml:

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <version>42.7.8</version>
</dependency>
```

La versión es un ejemplo; puedes utilizar una versión compatible más reciente.

4. Una particularidad importante de PostgreSQL

PostgreSQL distingue entre procedimientos y funciones:

- CREATE PROCEDURE: se invoca mediante CALL.
- CREATE FUNCTION: se invoca mediante una consulta SELECT cuando corresponde.

Anexo. Creación de la base de datos y Procedimiento desde el gestor de BD.

Aquí tienes el script SQL completo para crear la base de datos empresa en PostgreSQL, compatible con el procedimiento aumentar_salario que hemos preparado para JDBC.

Incluye la tabla empleados, claves primarias, restricciones y datos de prueba.

A1. Crear la base de datos

Ejecuta primero este comando desde pgAdmin o psql:

```
CREATE DATABASE empresa;
```

Después, conéctate a la base de datos empresa y ejecuta el resto del script.

A2. Crear la tabla empleados

```
CREATE TABLE empleados (
  emp_id INTEGER GENERATED ALWAYS AS IDENTITY,
  nombre VARCHAR(50) NOT NULL,
  apellidos VARCHAR(100) NOT NULL,
  departamento VARCHAR(50),
  salario NUMERIC(10, 2) NOT NULL,
  CONSTRAINT pk_empleados
    PRIMARY KEY (emp_id),
  CONSTRAINT chk_salario
    CHECK (salario >= 0)
);
```

Estructura de la tabla

Campo	Tipo	Descripción
emp_id	INTEGER	Identificador autogenerado y clave primaria
nombre	VARCHAR(50)	Nombre del empleado
apellidos	VARCHAR(100)	Apellidos del empleado
departamento	VARCHAR(50)	Departamento al que pertenece
salario	NUMERIC(10,2)	Salario con dos decimales

A3. Insertar datos de prueba

```
INSERT INTO empleados
  (nombre, apellidos, departamento, salario)
VALUES
  ('Ana', 'García López', 'Informática', 25000.00),
  ('Luis', 'Pérez Martín', 'Administración', 28000.00),
  ('María', 'Sánchez Ruiz', 'Informática', 32000.00),
  ('Carlos', 'Fernández Díaz', 'Recursos Humanos', 27000.00),
  ('Elena', 'Romero Torres', 'Informática', 35000.00);
```

PostgreSQL asignará automáticamente los identificadores a los empleados.

Puedes consultar los datos con:

```
SELECT *
FROM empleados
ORDER BY emp_id;
```

A4. Crear el procedimiento almacenado

Este procedimiento es compatible con la estructura anterior:

```
CREATE OR REPLACE PROCEDURE aumentar_salario(
  IN p_emp_id INTEGER,
  IN p_porcentaje NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
  IF p_porcentaje IS NULL OR p_porcentaje <= 0 THEN
    RAISE EXCEPTION
      'El porcentaje debe ser mayor que cero.';
  END IF;

  UPDATE empleados
  SET salario = salario * (1 + p_porcentaje / 100)
  WHERE emp_id = p_emp_id;

  IF NOT FOUND THEN
    RAISE EXCEPTION
      'Empleado no encontrado.';
  END IF;
END;
$$;
```

A5. Probar el procedimiento desde SQL

Por ejemplo, para aumentar un 10 % el salario del empleado con identificador 1:

```
CALL aumentar_salario(1, 10);
```

Comprueba el resultado:

```
SELECT emp_id, nombre, apellidos, salario
FROM empleados
WHERE emp_id = 1;
```

El salario de Ana pasará de 25.000 € a 27.500 €.

También puedes comprobar el tratamiento de errores:

```
-- Error: porcentaje no válido
CALL aumentar_salario(1, -5);


-- Error: empleado inexistente
CALL aumentar_salario(999, 10);
```

A6. Orden recomendado para la práctica de JDBC

1. Crear la base de datos empresa.
2. Crear la tabla empleados.
3. Insertar los cinco empleados.
4. Crear el procedimiento desde Java usando Statement.
5. Ejecutarlo mediante CallableStatement.
6. Consultar los resultados y gestionar las excepciones.

Nota: el usuario de PostgreSQL que utilices desde Java debe tener permisos para crear procedimientos y modificar la tabla. Además, si quieres que los errores de

las pruebas no dejen cambios parciales, gestiona las transacciones desde JDBC cuando desactives autoCommit.